Automotive Secure Code Review Assistant Technical Report

Karthikeya Bellapukonda, CB.AI.U4AID23109, Amrita Vishwa Vidyapeetham. Individual TechPulse Digital Track capstone, Case Study 4. Version 2.0, 12 October 2026.

Purpose and scope

This project helps an engineer review embedded C and C++ source before human approval. It accepts a source snippet or a repository ZIP, identifies a defined set of deterministic review candidates, maps repository structure, and retrieves relevant advisory guidance. It records review decisions and exports structured evidence. The October revision adds 13 deterministic rule families to the original seven, retrieval evidence for each candidate, a public-repository robustness benchmark, controlled mutations in actual ECU source, and a redesigned review workspace.

The central result is reproducible evidence rather than a claim that all software defects can be found. The public benchmark exercises 100 distinct source trees. A separate labelled mutation study tests four known defect families in real ECU functions. Independent retrieval queries compare lexical retrieval with hybrid retrieval. Actual local Ollama outputs demonstrate the generation stage. These experiments answer different questions and must remain separate.

System components

`run.py` starts a localhost FastAPI service and a Streamlit interface with matching API credentials. It finds free ports rather than terminating unrelated processes. The launcher's local profile authorizes `uploaded-repository` for a ZIP supplied by the user. Named demo repositories retain narrower path policies. Administrators can override this profile through environment variables.

FastAPI validates request types, limits text and archive sizes, authenticates API requests when configured, applies repository path permissions, invokes analysis in a thread pool, and persists the response through SQLite. Pydantic response models explicitly retain retrieval traces and citation validation, so evidence is not lost between the analyser and the browser. API integration tests also retrieve saved responses to confirm this preservation.

The repository analyser reads ZIP entries in memory. It does not extract repository content to a working tree and does not execute source, build files or scripts. It rejects traversal paths, absolute paths, drive-qualified paths, symbolic links, encrypted entries, duplicates, unsupported encodings and size violations. Its normal upload limits remain 8 MB compressed, 300 supported files, 256 KB per file, and 2 MB expanded source. Supported extensions are `.c`, `.cc`, `.cpp`, `.cxx`, `.h`, and `.hpp`. `.ino` and other extensions are outside the current analyser scope.

For each supported source, the analyser records functions, branch and loop counts, calls and quoted includes. It resolves internal include dependencies where the path is unambiguous. Missing include resolutions are evidence of an incomplete map, not proof of a missing runtime dependency. Macro expansion, templates and build configurations can alter the actual program and are not fully resolved by this text-based inventory.

The storage layer keeps review outputs and decisions in a local SQLite database. Source bodies are not deliberately persisted as separate source records, but findings contain short source-line evidence. Consequently, reports and the database can still contain code fragments and should be treated as sensitive in a real deployment. This distinction matters when discussing privacy.

Deterministic analysis

A deterministic check produces the same result for the same input and implementation. These checks use explicit patterns and limited local state rather than a language model. Each candidate records its title, severity, category, line, function, source evidence, remediation and a stable identifier. New rules also expose an `AUTO-*` rule ID.

Before matching, a lexical masker replaces comments and string or character literals with spaces while preserving offsets and newlines. This prevents a comment such as `// strcpy(dst, src)` from becoming a defect. Keeping offsets preserves the correct source line. The original source line is restored in the finding so a reviewer sees useful evidence. A regex lexer is still not a complete C++ lexer or preprocessor, particularly for raw string literals and conditional compilation.

The original checks cover unbounded input, unbounded string copy, concatenation and formatting, explicit null-pointer dereference, potentially unreleased allocation, and switch statements without a default. The new checks cover literal zero division/modulo, constant array indexes beyond a local fixed declaration, invalid fixed-width shifts, assignment in an `if`, nonliteral `printf` formats, repeated `free`, pointer access after `free`, allocation or blocking in interrupt-style functions, direct recursion, classical CAN DLC above eight, empty unbounded busy waits, and shell execution.

Some candidates identify a direct syntactic hazard, while others identify a need for engineering review. Allocation in an ISR-named function is a latency concern, but the function's name alone cannot prove it runs in interrupt context. A missing default branch may be intentional for an exhaustive enum. A malloc result may transfer ownership to a caller. The UI therefore reports candidates rather than certified violations.

The shift rule assumes a 32-bit `int` target and accounts for integer promotion by using at least 32 bits for small integer types. A 16-bit MCU target needs a different explicit target profile. The CAN rule deliberately targets `can_dlc`, associated with classical CAN. CAN FD supports larger payloads and uses different length/DLC semantics. The rule card exposes this assumption; a production profile must select the correct protocol and target architecture.

Pointer and resource reasoning is intentionally limited. It does not implement interprocedural alias analysis, a control-flow graph, ownership transfer proof or path-sensitive reachability. Same-line reassignment and cross-function pointer-name contamination are covered by regression tests, but branch-dependent allocation and release remain difficult. A clean result means no targeted pattern was found, not that the code is safe.

Retrieval augmented generation

RAG means retrieval augmented generation. Retrieval selects relevant material from a corpus. Generation uses that selected material to produce an answer. It avoids asking a model to recall a coding standard from training memory. In this implementation, retrieval also supports an extractive template mode when the local LLM is unavailable. The UI labels these modes separately.

The original version chunked a small mock standards PDF, embedded the chunks and stored them in Chroma. Its review query mixed raw code, warnings, neighboring files and historical findings. Its citation check only checked whether a model's reference text appeared somewhere in the context. That could demonstrate neither direct applicability nor reliable citation support.

The active version 2 retrieval path uses 20 versioned advisory rule cards. Each card has an ID, text, category, domain, language, version, origin and assumptions. They are explicitly synthetic project guidance. They are not official MISRA or ISO clauses, and their IDs must never be presented as such. The legacy PDF ingestion utilities remain available for experiments, but the active rule-card review does not consult their Chroma collection or claim licensed standards coverage.

The corpus is exported as `data/standards/automotive_rule_cards.json`. The source generator is `tools/build_rule_corpus.py`. A corpus hash identifies the exact knowledge version. Each retrieved card also carries a hash. These hashes detect a change in bytes or metadata; they do not prove that the rule text is correct or authorized.

Automotive evidence gated retrieval architecture

The project-specific architectural contribution is a retrieval pipeline organized around explicit candidate evidence. This is an engineering design contribution, not a claim of a new scientific retrieval algorithm. BM25, sentence embeddings and reciprocal rank fusion are established techniques.

First, the deterministic detector supplies a defect title, category and recommended check. The retrieval query uses those fields instead of an entire source repository. This keeps long unrelated code and malicious source comments from dominating ranking. It also means the current pipeline principally grounds detected candidates. An unsupported semantic defect that the detector misses will not automatically receive guidance.

Second, BM25 scores lexical agreement with the rule cards. BM25 rewards a query term more when it is unusual across the corpus, accounts for repeated terms, and normalizes document length. It preserves exact embedded vocabulary such as `can_dlc`, `ISR`, `strcpy` and `uint32_t`. Pure lexical search can miss a paraphrase such as 'buffer accessed after release'.

Third, the locally cached `all-MiniLM-L6-v2` sentence transformer embeds queries and rule texts. Embeddings are numeric vectors representing linguistic similarity. Normalized vectors allow a dot product to act as cosine similarity. Dense retrieval can connect 'release' with 'free' even without the exact same word. Similarity can also retrieve a nearby but inapplicable rule, so it cannot authorize a citation by itself.

Fourth, reciprocal rank fusion combines lexical and dense rankings. For each candidate document, the score is the sum of `1 / (60 + rank)` over available ranked lists, with ranks beginning at one. Fusion avoids assuming that a BM25 score and a cosine score share a meaningful scale. When the model is unavailable, the strategy explicitly becomes BM25-only. The trace records the actual strategy and dense error rather than labelling the fallback hybrid.

Fifth, an applicability gate checks the detector's typed rule ID against the retrieved top-five cards. A semantically related card cannot substitute for a missing typed rule. A match yields `supported-candidate`; no match yields `abstain`. This confirms relevance of advisory guidance, not truth of the source-code defect. Deleting a relevant card must change support to abstention while leaving the detector's candidate intact. The deletion experiment proves retrieval is contributing information.

Sixth, the local `llama3.1` model receives applicable cards plus bounded, escaped source and evidence. The prompt asks for structured JSON and exact quoted evidence. The response passes a programmatic citation gate: the rule ID must identify an applicable retrieved card, and each evidence item must be a nonempty extract from matching context. Invented IDs, unrelated IDs, fabricated quotes and empty evidence fail closed. Rejection clears the model citation, evidence and remediation, and lowers confidence. This is stronger than relying on prompt wording alone, although extract matching still does not prove every model sentence is semantically correct.

Finally, the API, saved report and UI expose the query, top candidates, lexical and dense scores, fusion score, applicability decision, rule-card hash, corpus hash and citation-validation decision. A reviewer can inspect how an answer was produced without requesting hidden model reasoning. The architecture creates a trace of observable evidence, not a chain-of-thought transcript.

Why these design choices suit automotive reviews

Automotive source contains protocol fields, hardware types, short identifiers and safety assumptions that generic prose search can overlook. Exact lexical matching preserves those terms while dense retrieval handles the engineer's paraphrases. Typed applicability prevents a retrieved rule on an adjacent topic from becoming a purported safety claim. Explicit assumptions make the target-width and CAN/CAN FD boundary visible.

Local execution keeps source away from a hosted model endpoint in the supplied configuration. It does not by itself provide enterprise access controls or encryption. The approved model must already exist locally; the review path does not download it. Corpus changes invalidate retrieval caches through content-based cache keys. Cached rule embeddings and repeated query results reduce repeated inference work. The active card index is small and in memory; a large licensed standards corpus would require metadata filtering, a persistent index and measured retrieval latency.

A stronger future version should accept a reviewed ECU profile specifying MCU integer width, CAN/CAN FD, RTOS/ISR contracts, applicable standard edition and permitted allocation policy. A compiler AST and control-flow graph should provide typed code facts. Retrieval could then filter by profile before ranking and attach each cited requirement to a specific code fact or test. That extension is proposed, not implemented or validated here. Hardware-in-the-loop and worst-case execution-time measurement also remain future work.

Verification strategy

The evidence has four layers. Component tests verify specific contracts, including input validation, ZIP boundaries, API authentication, persistence, detector positives and corrected counterexamples. The public benchmark verifies robustness and scope across actual source trees. ECU mutations verify known defects inserted into real functions. RAG experiments verify ranking, applicability, abstention and actual generation. None can replace the others.

The public benchmark downloads read-only GitHub snapshots and never executes their contents. A catalog records repository identity and discovery query. Each record includes the snapshot root, archive SHA-256, per-file hashes, exclusions, findings and elapsed time. Large source trees are divided into batches within the production scanner's file and expanded-text limits. This exercises the actual ZIP analyser on all eligible sources, but cross-partition include relationships are incomplete. It does not demonstrate that one oversized archive can bypass the application's upload limits.

The selected corpus is a public convenience sample of automotive and related embedded projects, including tutorials and prototypes. It is not a random sample of production ECUs. The first run scanned 100 distinct repositories, 6,721 eligible files and 2,650,420 lines with no repository-level failures. Candidate totals and final implementation hashes are recorded in the regenerated benchmark summary. Counts describe parser coverage, not verified vulnerabilities. Timings include cache/download and scan effects and are not ECU execution times or isolated analyser throughput.

Real ECU mutation experiment

Speeduino and rusEFI are actual open-source engine-control projects. The experiment selects five real source files from each project. Speeduino's snapshot root identifies revision `b39d27c`; the rusEFI sources use commit `0a4b38b9250a981261dc59658b56794dee0e1567`. The provenance file records archive and source hashes and preserves each upstream license in the manual ZIP fixtures. Sources: https://github.com/speeduino/speeduino and https://github.com/rusefi/rusefi.

For each selected function, the harness inserts a clearly named educational statement immediately after its opening brace. The four families are literal zero division, explicit null dereference, constant out-of-bounds array access and unbounded copy. Baseline and mutant candidate counts are recorded, and detection requires the expected title at the injected line. This prevents an unrelated pre-existing candidate from counting as mutation detection.

All 40 controlled mutations were detected. This 40/40 result applies to the selected examples of those four supported families. It does not measure performance on arbitrary real-world defects. These fixtures contain injected defects, and the project makes no assertion that the upstream authors introduced them. The full ECU firmware was not compiled, executed or flashed. Small baseline/mutant ZIPs provide manual upload examples, while patches cover all cases.

RAG experiments and their interpretation

Twenty manually labelled natural-language queries represent the 20 advisory rules. These queries are separate from the detector's generated query format. Lexical retrieval achieved recall@5 of 0.90, top-one accuracy of 0.70 and MRR@5 of approximately 0.792. Hybrid BM25 plus actual local embeddings achieved recall@5 of 0.95, top-one accuracy of 0.85 and MRR@5 of approximately 0.892. The JSON evidence preserves the missed and lower-ranked cases.

Recall@5 is the fraction of queries whose labelled relevant rule appears in the first five results. Top-one accuracy is the fraction ranked first. Mean reciprocal rank averages `1 / rank` for a relevant retrieved item and uses zero if it is absent from the top five. These are retrieval measurements on a small author-labelled corpus. They are not model accuracy on real code, and one extra correct query changes recall by five percentage points.

Four intervention controls passed: the correct rule supplies support, deleting it causes abstention, replacing the corpus with an unrelated rule causes abstention, and a forged model citation is rejected. Additional automated tests reject empty quotes, unknown IDs and unrelated evidence. These controls show that a successful result depends on the corpus and validation logic.

`prove_live_rag.py` calls the production orchestrator with the actual installed local `llama3.1` model. Both defect examples exercised actual generation. In the final recorded run, the unsafe-copy answer passed citation validation, while the zero-division answer failed validation and its unsupported model evidence was rejected. The separately retrieved rule still supports the deterministic zero-division

candidate. A clean arithmetic comparison returned `deterministic-only` without claiming an applicable rule. A mock-LLM unit test is not counted as actual generation evidence. Actual outcomes, including rejected proposals, remain inspectable rather than assuming every generated answer passes.

Scalability and operational boundaries

The rejected zero-division answer put the C source line in its evidence field rather than quoting the retrieved advisory text. The validator rejected that proposal while retaining the separately grounded deterministic finding. This is a concrete example of why a model's plausible rule ID is insufficient without supporting retrieved evidence.

The 100-repository corpus demonstrates that the scanner handles varied source layouts and millions of lines across bounded batches. It does not prove concurrent enterprise service capacity. The Streamlit UI pages findings in groups of 20 and offers severity, category and text filters, while the JSON export retains the full result. Retrieval traces displayed inline are bounded, with full traces available in exported reports.

Useful additional performance work would measure analyser-only time, peak memory, simultaneous requests, cold versus warm embedding latency and model throughput. Those should use fixed hardware, pinned input snapshots and a declared concurrency level. No such unmeasured capacity numbers are claimed. The current file limits and highest-risk-module LLM review remain deliberate service boundaries.

Security and human review

Source code is untrusted text. Source comments cannot override the review instructions, and retrieved data is escaped into prompt delimiters. The query uses detector facts rather than copying arbitrary source comments. This reduces prompt-injection opportunities but cannot establish a universal prompt-injection guarantee. Corpus administration remains a trusted responsibility.

API keys use constant-time comparison. Path authorization controls which source prefixes may be analysed in a configured repository. These controls are demonstrated by positive and negative tests, but they are not enterprise identity management, project-level encryption or an independently audited multi-tenant design. Human decisions remain recorded as accepted, rejected or needing changes. Neither an LLM response nor the absence of findings approves a release.

Reproducing and presenting the evidence

Run `python evaluation/component_verification.py` for component evidence, CLI gates and an independent GCC warning where GCC is installed. Run `python evaluation/prove_rag.py` for retrieval comparisons and controls. Run `python evaluation/prove_live_rag.py` with Ollama running for actual local generation. Run `python evaluation/ecu_mutation_study.py` for ECU fixtures and mutation results. Run `python evaluation/benchmark_public_repos.py --target 100 --workers 6` for the public corpus. The first public and ECU runs require network access to obtain upstream sources; normal code review remains local.

For a presentation, begin with one real ECU baseline/mutant pair. Show the added line and matching finding. Open the retrieved guidance and its assumption, then inspect the query and scores. Next, show the deleted-rule control to explain abstention. Finish with the public corpus summary and its scope limitations. Download the structured report so the evaluator can inspect evidence independently.

Oral examination questions

Why combine deterministic checks with RAG? The checks give repeatable, located candidates. Retrieval supplies inspectable guidance. A local model explains that guidance, while programmatic validation limits unsupported citations. Each part has a distinct contract and separate tests.

Why use both BM25 and embeddings? Exact embedded identifiers benefit from lexical matching, while paraphrased engineering language benefits from semantic similarity. The measured comparison improved recall@5 from 18/20 to 19/20 on the labelled query set. A larger independent dataset is required to establish generalization.

Why does an applicability gate remain necessary after good retrieval? High similarity does not imply direct relevance. A null-pointer rule can be semantically close to a use-after-free defect but cannot replace its missing guidance. The gate uses a typed detector rule ID and abstains when that ID is absent from the retrieved evidence.

What proves generation actually ran? The saved live evidence records actual local model availability, execution mode, elapsed time, retrieved context, returned JSON and citation-validation outcome from the production orchestrator. A fallback is labelled as a fallback and is not counted as model generation.

Does a 100-repository scan prove 100 percent accuracy? No. It demonstrates robustness across those eligible files. Precision requires confirmed labels for reported candidates. Recall requires known defects, including defects the analyser misses. The mutation study supplies limited labels for four families, and corrected synthetic counterexamples check specific false-positive risks.

Are the rules genuine MISRA clauses? The active 20 cards are synthetic advisory rules with project IDs. Licensed, edition-specific MISRA material would need approved ingestion, provenance and applicability review before making standards claims. A mock PDF or plausible rule number does not establish compliance.

What is the largest architectural limitation? The analyser uses text patterns and limited local state instead of a full compiler representation. Therefore it cannot prove path feasibility, aliases, macro-expanded behaviour or system timing. Compiler AST integration and reviewed ECU profiles are the next useful improvements.

What would you defend as the contribution? A locally runnable, auditable review assistant with typed candidate-to-guidance applicability, hybrid retrieval, explicit abstention, citation validation, inspectable traces and a reproducible multi-layer evidence suite. The retrieval algorithms themselves are established, and production safety certification is outside the capstone's demonstrated scope.

Measured evidence snapshot

Component verification

93 of 93 automated tests passed, with 0 failures and 0 skips. Both actual CLI severity-gate comparisons passed. The JSON and JUnit XML preserve each case and implementation hashes.

Evidence: Evaluation_Results/component_evidence.json

Public corpus

100 distinct repositories, 6,721 eligible C/C++ files and 2,650,420 lines scanned. Repository-level failures: 0. Candidate findings: 2,017. Median total cache/download-plus-scan time: 0.679 seconds per repository. This is not isolated analyzer latency.

Evidence: Evaluation_Results/public_benchmark/summary.json

ECU mutation study

40 of 40 injected cases detected across 10 actual files from 2 upstream projects. The four mutation families and injected-line checks are inspectable in the per-case records.

Evidence: Evaluation_Results/ecu_mutations/results.json

Retrieval experiment

20 author-labelled queries. Lexical recall at 5: 90%, top-one accuracy: 70%, MRR at 5: 0.792. Hybrid recall at 5: 95%, top-one accuracy: 85%, MRR at 5: 0.892. All four corpus/citation intervention controls passed. Actual hybrid strategy: BM25 plus dense reciprocal rank fusion.

Evidence: Evaluation_Results/rag_evidence/retrieval_results.json